LABORATORIO #5

API REST con Express y Swagger integrando la Claude API

Guía de trabajo en parejas · Programación Backend

Modalidad	En parejas	**Valor**	100 puntos (+10 extra)
Duración sugerida	Una sesión de laboratorio (2–3 h) más trabajo en casa	**Fecha de entrega**	_____
Integrante A	_____	**Integrante B**	_____
Repositorio	https://github.com/_____		

1 Objetivos

Al finalizar el laboratorio, cada pareja será capaz de:

- Construir una API REST con Express que gestione un recurso (notas) con validación y manejo de errores centralizado.
- Integrar la Claude API desde el backend con el SDK oficial **@anthropic-ai/sdk**, usando una API key guardada en variables de entorno.
- Proteger credenciales: la API key nunca llega al cliente, a Git ni a capturas de pantalla.
- Documentar todos los endpoints con OpenAPI 3 y probarlos desde Swagger UI.
- Trabajar en equipo con Git: ramas, pull requests y revisión de código entre compañeros.

2 Qué van a construir

Una API de **notas** con dos funciones de inteligencia artificial: responder preguntas libres y resumir el contenido de una nota. El cliente (Swagger UI o Postman) solo habla con su API; **solo el backend habla con Claude**. Las notas se guardan en memoria para centrar el trabajo en la integración.

Método	Endpoint	Descripción	Usa Claude	Responsable
GET	`/api/notas`	Lista todas las notas	No	A
GET	`/api/notas/:id`	Obtiene una nota	No	A
POST	`/api/notas`	Crea una nota (titulo, contenido)	No	A
DELETE	`/api/notas/:id`	Elimina una nota	No	A
POST	`/api/ia/preguntar`	Envía una pregunta libre al modelo	Sí	B
POST	`/api/notas/:id/resumen`	Resume una nota en 3 viñetas	Sí	B
GET	`/api-docs`	Documentación interactiva (Swagger UI)	No	A

3 Organización del trabajo en parejas

Cada integrante asume un rol principal, pero **ambos deben entender y poder explicar todo el proyecto**. El docente puede preguntar a cualquiera de los dos por cualquier parte del código.

Rol	Responsabilidades	Archivos principales
Integrante A API y documentación	Configurar Express y Swagger, CRUD de notas, validación con Zod, documentar los endpoints de notas.	`app.js, index.js, config/swagger.js, notas.routes.js, notas.service.js, validar.js`
Integrante B Integración con IA	Obtener la API key, servicio de Claude, endpoints de IA, errores de la Claude API, documentar los endpoints de IA.	`claude.service.js, ia.routes.js, ruta /:id/resumen, errorHandler.js`
Ambos	Crear el repositorio, configurar .env, pruebas cruzadas, README y respuestas de reflexión.	`.env.example, .gitignore, README.md`

Reglas de colaboración

- Un solo repositorio en GitHub con ambos integrantes como colaboradores.
- Cada rol trabaja en su rama: `feature/notas` (A) y `feature/ia` (B). Nada se sube directo a `main`.
- Todo cambio entra por **pull request** revisado y aprobado por la pareja (deben quedar comentarios de revisión).
- Cada integrante debe tener al menos **5 commits significativos** con mensajes descriptivos.
- Punto de integración: el endpoint de resumen (B) usa `notas.service` (A). Acuerden la función `buscar(id)` antes de empezar.

API key compartida: cada integrante la guarda en su propio archivo .env local. Si la comparten, háganlo por un canal privado, nunca por el chat del grupo, el repositorio ni capturas de pantalla. Si se filtra, bórrenla de inmediato en la Claude Console y creen otra.

4 Requisitos previos

- Node.js 20 o superior (`node -v`), Git y una cuenta de GitHub por integrante.
- Cuenta en la Claude Console (platform.claude.com) con créditos disponibles, o la API key que entregue el docente.
- VS Code y, opcionalmente, Postman o Thunder Client.
- Haber completado los laboratorios de REST, Express y middleware/validación/errores.

Costo: cada llamada a la Claude API consume créditos. Usen el modelo indicado en el .env, prompts cortos y no ejecuten pruebas en bucle.

5 Desarrollo paso a paso

PASO 1 Crear el repositorio y el proyecto

Responsable: Ambos

El integrante A crea el repositorio en GitHub e invita a B como colaborador. Ambos lo clonan y A ejecuta:

Terminal

```
$ npm init -y
$ npm install express @anthropic-ai/sdk dotenv zod swagger-jsdoc swagger-ui-express
$ npm install -D nodemon
```

En `package.json` agreguen los scripts `"start": "node src/index.js"` y `"dev": "nodemon src/index.js"`.

Paquete	Para qué sirve
`express`	Servidor HTTP y rutas (se instala Express 5).
`@anthropic-ai/sdk`	SDK oficial para llamar a la Claude API.
`dotenv`	Carga las variables del archivo .env en process.env.
`zod`	Validación de los datos que llegan en req.body.
`swagger-jsdoc / swagger-ui-express`	Generan la especificación OpenAPI y la interfaz /api-docs.

PASO 2 Crear la estructura de carpetas

Responsable: Integrante A

Estructura

```
lab5-api/
├── src/
│   ├── config/swagger.js
│   ├── routes/notas.routes.js
│   ├── routes/ia.routes.js
│   ├── services/notas.service.js
│   ├── services/claude.service.js     ← único archivo que usa el SDK
│   ├── middlewares/validar.js
│   ├── middlewares/errorHandler.js
│   ├── utils/errors.js
│   ├── app.js
│   └── index.js
├── .env              ← API key (NO se sube)
├── .env.example
├── .gitignore
├── README.md
└── package.json
```

PASO 3 Obtener la API key

Responsable: Integrante B

- Entrar en **platform.claude.com** e iniciar sesión (o crear una cuenta).
- Ir a **Settings → API keys** y pulsar **Create key**. Nómbrenla, por ejemplo, *lab5-pareja-N*.
- Copiarla de inmediato: empieza por `sk-ant-` y **solo se muestra una vez**. Si se pierde, se crea otra.

Si el docente entrega la clave, omitan este paso y usen la que reciban. El tratamiento de seguridad es exactamente el mismo.

PASO 4 Variables de entorno y .gitignore

Responsable: Ambos

Cada integrante crea su propio archivo .env en la raíz (nunca se sube a GitHub):

.env

```
ANTHROPIC_API_KEY=sk-ant-api03-...
CLAUDE_MODEL=claude-haiku-4-5-20251001
PORT=3000
```

.gitignore

```
node_modules/
.env
```

.env.example (sí se sube, sin la clave)

```
ANTHROPIC_API_KEY=
CLAUDE_MODEL=claude-haiku-4-5-20251001
PORT=3000
```

El SDK lee `ANTHROPIC_API_KEY` automáticamente, por eso la clave nunca aparece en el código. `CLAUDE_MODEL` permite cambiar de modelo sin modificar archivos.

Comprobación obligatoria: antes del primer commit ejecuten `git status` y verifiquen que .env no aparece en la lista.

PASO 5 Servidor base: app.js, index.js, Swagger y utilidades

Responsable: Integrante A

src/utils/errors.js

```
class AppError extends Error {
  constructor(status, message) {
    super(message);
    this.status = status;
  }
}
module.exports = { AppError };
```

src/middlewares/validar.js

```
module.exports = (schema) => (req, res, next) => {
  const r = schema.safeParse(req.body);
  if (!r.success)
    return res.status(400).json({ errores: r.error.issues });
  req.body = r.data; // datos limpios
  next();
};
```

src/middlewares/errorHandler.js (versión inicial; B la amplía en el paso 10)

```
const { AppError } = require('../utils/errors');

module.exports = (err, req, res, next) => {
  if (err instanceof AppError)
    return res.status(err.status).json({ error: err.message });

  console.error(err);
  res.status(500).json({ error: 'Error interno' });
};
```

src/config/swagger.js

```
const swaggerJsdoc = require('swagger-jsdoc');

module.exports = swaggerJsdoc({
  definition: {
    openapi: '3.0.0',
    info: { title: 'Lab 5: Notas + IA', version: '1.0.0' },
    servers: [{ url: 'http://localhost:3000' }],
    components: {
      schemas: {
        Pregunta: {
          type: 'object',
          required: ['pregunta'],
          properties: { pregunta: { type: 'string' } },
        },
      },
    },
  },
  apis: ['./src/routes/*.js'], // dónde leer los @openapi
});
```

src/app.js

```
const express = require('express');
const swaggerUi = require('swagger-ui-express');
const spec = require('./config/swagger');
const errorHandler = require('./middlewares/errorHandler');

const app = express();
app.use(express.json());

app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(spec));
app.use('/api/notas', require('./routes/notas.routes'));
app.use('/api/ia', require('./routes/ia.routes'));

app.use(errorHandler); // siempre al final
module.exports = app;
```

src/index.js

```
require('dotenv').config();
const app = require('./app');

const PORT = process.env.PORT || 3000;
app.listen(PORT, () =>
  console.log(`API en http://localhost:${PORT}/api-docs`));
```

Clave: `require('dotenv').config()` va en la primera línea de index.js, antes de importar cualquier archivo que use process.env.

Para que arranque ya, creen `notas.routes.js` e `ia.routes.js` exportando un router vacío. Ejecuten `npm run dev` y abran `http://localhost:3000/api-docs`.

PASO 6 CRUD de notas

Responsable: Integrante A

src/services/notas.service.js

```
const { AppError } = require('../utils/errors');

let notas = [];
let nextId = 1;

const listar = () => notas;

function buscar(id) {
  const n = notas.find((n) => n.id === id);
  if (!n) throw new AppError(404, 'Nota no encontrada');
  return n;
}

function crear({ titulo, contenido }) {
  const n = { id: nextId++, titulo, contenido };
  notas.push(n);
  return n;
}

// TODO (A): function eliminar(id) { ... }  // usa buscar() para el 404

module.exports = { listar, buscar, crear };
```

src/routes/notas.routes.js

```
const router = require('express').Router();
const { z } = require('zod');
const validar = require('../middlewares/validar');
const notas = require('../services/notas.service');

const notaSchema = z.object({
  titulo: z.string().min(1).max(100),
  contenido: z.string().min(1).max(5000),
});

router.get('/', (req, res) => res.json(notas.listar()));

router.post('/', validar(notaSchema), (req, res) =>
  res.status(201).json(notas.crear(req.body)));

// TODO (A): GET /:id    -> 200 con la nota o 404
// TODO (A): DELETE /:id -> 204 o 404

module.exports = router;
```

A completa los TODO: la función `eliminar(id)` en el servicio y las rutas `GET /:id` y `DELETE /:id`. Recuerden que `req.params.id` llega como texto: conviértanlo con `Number()`. Con Express 5, un error lanzado dentro de la ruta llega solo al errorHandler.

PASO 7 Servicio de Claude

Responsable: Integrante B

src/services/claude.service.js

```
const Anthropic = require('@anthropic-ai/sdk');
const client = new Anthropic(); // lee ANTHROPIC_API_KEY

async function preguntar(texto, system) {
  const msg = await client.messages.create({
    model: process.env.CLAUDE_MODEL,
    max_tokens: 1024,
    system: system ?? 'Eres un asistente útil. Responde en español.',
    messages: [{ role: 'user', content: texto }],
  });
  const bloque = msg.content.find((b) => b.type === 'text');
  return { respuesta: bloque?.text ?? '', uso: msg.usage };
}

module.exports = { preguntar };
```

Parámetro	Qué hace
`model`	Modelo a usar; viene del .env.
`max_tokens`	Límite de tokens de la respuesta: controla la longitud y el costo.
`system`	Instrucciones de comportamiento del modelo (rol, idioma, formato).
`messages`	La conversación; aquí un solo mensaje con rol user.
`msg.content`	Lista de bloques de la respuesta; se toma el primero de tipo text.
`msg.usage`	Tokens de entrada y de salida consumidos en la llamada.

PASO 8 Endpoint POST /api/ia/preguntar

Responsable: Integrante B

src/routes/ia.routes.js

```
const router = require('express').Router();
const { z } = require('zod');
const validar = require('../middlewares/validar');
const claude = require('../services/claude.service');

const preguntaSchema = z.object({
  pregunta: z.string().min(3).max(2000),
});

router.post('/preguntar', validar(preguntaSchema), async (req, res) => {
  const r = await claude.preguntar(req.body.pregunta);
  res.json(r);
});

module.exports = router;
```

Validar antes de llamar a Claude evita gastar créditos en peticiones vacías o enormes; el max(2000) también limita el costo de entrada.

PASO 9 Endpoint POST /api/notas/:id/resumen (integración A + B)

Responsable: Integrante B

B añade esta ruta en `notas.routes.js` usando el servicio de A. Es el punto donde se unen las dos ramas: resuelvan juntos los conflictos del pull request.

src/routes/notas.routes.js (agregar)

```
// Al inicio del archivo:
const claude = require('../services/claude.service');

router.post('/:id/resumen', async (req, res) => {
  const nota = notas.buscar(Number(req.params.id)); // 404 si no existe
  const r = await claude.preguntar(
    nota.contenido,
    'Resume el texto en 3 viñetas breves, en español.'
  );
  res.json({ notaId: nota.id, resumen: r.respuesta });
});
```

Experimenten cambiando el texto de `system`: «resume en una frase», «extrae las palabras clave», «traduce al inglés». Anoten en el README qué cambió.

PASO 10 Manejar los errores de la Claude API

Responsable: Integrante B

src/middlewares/errorHandler.js (ampliar)

```
const Anthropic = require('@anthropic-ai/sdk');   // al inicio

  // Dentro del manejador, ANTES del console.error(err) final:
  if (err instanceof Anthropic.APIError) {
    console.error('Claude API:', err.status, err.message);
    const status = err.status === 429 ? 429
      : err.status === 529 ? 503 : 502;
    return res.status(status).json({ error: 'IA_NO_DISPONIBLE' });
  }
```

Status de Claude	Tipo	Causa habitual	Su API responde
401	authentication_error	API key ausente o inválida	502 sin detalles
400	invalid_request_error	Modelo o parámetros incorrectos	502
429	rate_limit_error	Demasiadas peticiones	429
529	overloaded_error	Servicio saturado temporalmente	503

Nunca reenvíen el mensaje original de Claude al cliente: regístrenlo en la consola del servidor y respondan un error genérico. El SDK ya reintenta por su cuenta algunos fallos temporales.

PASO 11 Documentar los endpoints con @openapi

Responsable: Ambos

Cada integrante documenta sus propios endpoints: A los de notas y B los de IA. El comentario va justo encima de la ruta y es YAML, así que **la sangría importa**. Ejemplo de la ruta de IA:

`Comentario encima de router.post('/preguntar', ...)`

```
/**
 * @openapi
 * /api/ia/preguntar:
 *   post:
 *     summary: Envía una pregunta a Claude
 *     tags: [IA]
 *     requestBody:
 *       required: true
 *       content:
 *         application/json:
 *           schema:
 *             $ref: '#/components/schemas/Pregunta'
 *     responses:
 *       200: { description: Respuesta del modelo }
 *       400: { description: Datos inválidos }
 *       502: { description: Claude no disponible }
 */
```

- Documenten los **7 endpoints**, incluidos los códigos de error (400, 404, 502), no solo el 200.
- Agrupen con `tags`: *Notas* e *IA*. Agreguen un esquema `Nota` en `components.schemas` para el body de POST /api/notas.
- Si un endpoint no aparece en /api-docs, revisen la sangría y la ruta `apis` de swagger.js.

PASO 12 Pruebas cruzadas desde Swagger UI

Responsable: Ambos

A prueba los endpoints de B y B prueba los de A, desde `/api-docs` con *Try it out* → *Execute*. Marquen cada caso y guarden captura de los marcados con ★.

	Caso de prueba	Resultado esperado	Lo prueba
☐	Crear dos notas con POST /api/notas	201 con id	**B**
☐	Listar con GET /api/notas	200 con las dos notas	**B**
☐	Pedir GET /api/notas/99	404 Nota no encontrada	**B**
☐	POST /api/notas con body vacío {}	400 con detalles de Zod	**B**
☐	Pregunta libre en POST /api/ia/preguntar ★	200 con respuesta y uso de tokens	**A**
☐	Resumir la nota 1 con POST /api/notas/1/resumen ★	200 con resumen en 3 viñetas	**A**
☐	Resumir una nota que no existe	404 sin llamar a Claude	**A**
☐	POST /api/ia/preguntar con pregunta de 1 letra	400 sin llamar a Claude	**A**
☐	Cambiar la API key por una falsa y preguntar ★	502 IA_NO_DISPONIBLE; 401 en la consola	**A**
☐	DELETE /api/notas/2 y luego GET /api/notas/2	204 y después 404	**B**

PASO 13 README y respuestas de reflexión

Responsable: Ambos

El README.md debe incluir: nombres de los integrantes y su rol, pasos para instalar y ejecutar, la lista de variables de entorno (sin valores), capturas de Swagger UI y las respuestas a la sección 6.

6 Preguntas de reflexión (responder en el README)

1. ¿Por qué la API key no puede estar en el frontend? Describan cómo la podría obtener un usuario cualquiera.
2. ¿Qué responde su API cuando Claude devuelve 401? ¿Por qué no se reenvía ese mismo 401 al cliente?
3. ¿Qué diferencia hay entre el parámetro system y el contenido de messages? Den un ejemplo de su prueba.
4. Anoten los input_tokens y output_tokens de dos de sus llamadas. ¿Qué parámetro les permite limitar el costo?
5. ¿Qué pasaría si no validaran el body antes de llamar a Claude? Mencionen al menos dos consecuencias.
6. ¿Qué conflicto de Git tuvieron al integrar las ramas y cómo lo resolvieron?

7 Entregables

- Enlace al repositorio de GitHub con la rama `main` actualizada y los pull requests visibles.
- README.md completo según el paso 13.
- `.env.example` con los nombres de las variables, sin valores secretos.
- Capturas de los casos marcados con ★ (pueden ir en una carpeta docs/ del repositorio).
- Coevaluación individual: cada integrante entrega por separado el formulario que indique el docente.

8 Rúbrica de evaluación

Criterio	Descripción	Pts
CRUD de notas	Los 4 endpoints funcionan con los códigos de estado correctos (200, 201, 204, 404).	15
Validación y errores	Zod en notas e IA; errorHandler central con formato uniforme.	15
Integración con Claude	claude.service con messages.create; preguntar y resumir funcionan; uso de system.	25
Seguridad de la API key	.env fuera de Git, .env.example correcto, errores de Claude sin exponer detalles.	15
Documentación Swagger	Los 7 endpoints documentados con esquemas, tags y códigos de error.	15
Trabajo en pareja	Ramas, pull requests con revisión, commits de ambos, README y reflexión.	15
Total		**100**

Penalizaciones: si la API key aparece en cualquier commit del historial, el criterio de seguridad vale 0 y la clave debe rotarse. Un integrante sin commits ni pull requests propios no recibe los puntos de trabajo en pareja.

9 Retos extra (hasta +10 puntos, 5 cada uno)

Reto	Descripción
Conversación	Guardar el historial y enviarlo en messages para chatear en varios turnos.
Streaming	Usar `client.messages.stream()` y enviar el texto al cliente a medida que llega.
Rate limit	Proteger /api/ia con `express-rate-limit` para que nadie gaste sus créditos.
Persistencia	Reemplazar el arreglo de notas por Prisma con SQLite.
Pruebas automatizadas	Tests con supertest simulando claude.service, sin gastar créditos.

10 Recursos

- Cómo obtener la API key: platform.claude.com/docs/es/get-api-key
- SDK de TypeScript/JavaScript: platform.claude.com/docs/en/api/sdks/typescript
- Express 5: expressjs.com · Zod: zod.dev
- Especificación OpenAPI: swagger.io/specification · swagger-jsdoc en npm